1. What does a pointer variable store in the C programming language?

(A) Memory address of var
(B) Floating-point value
(C) ASCII code only
(D) Instruction opcode

2. Which operator is used to obtain the physical memory address of a variable?

(A) & (Address-of)
(B) * (Indirection)
(C) -> (Arrow)
(D) % (Modulo)

3. Which operator is used to access or modify the value stored at an address held by a pointer?

(A) * (Dereference)
(B) & (Address-of)
(C) sizeof
(D) -> (Arrow)

4. What is a pointer that has not been initialized to any valid memory address called?

(A) Wild pointer
(B) NULL pointer
(C) Dangling pointer
(D) Void pointer

5. What standard macro represents a pointer pointing to nothing / memory address 0?

(A) NULL
(B) NIL
(C) 0xFFFFFFFF
(D) EMPTY

6. What is the size of any pointer variable on a standard 64-bit microprocessor architecture?

(A) 8 bytes
(B) 4 bytes
(C) 2 bytes
(D) 16 bytes

7. What is the size of a char pointer (char*) on a 64-bit operating system?

(A) 8 bytes
(B) 1 byte
(C) 4 bytes
(D) 16 bytes

8. What is a generic pointer in C that can point to any data type called?

(A) void pointer (void*)
(B) generic pointer
(C) null pointer
(D) dynamic pointer

9. Can a void pointer (void*) be directly dereferenced without explicit type casting in ANSI C?

(A) No (must be cast)
(B) Yes (defaults to int)
(C) Only in GCC
(D) Only for char

10. If int *ptr = 1000 and sizeof(int) is 4, what is the address evaluated by (ptr + 1)?

(A) 1004
(B) 1001
(C) 1002
(D) 1008

11. If double *p = 2000 and sizeof(double) is 8, what address is evaluated by (p + 2)?

(A) 2016 (2000 + 2*8)
(B) 2002
(C) 2008
(D) 2032

12. What type does the difference between two pointers of the same type evaluate to?

(A) ptrdiff_t (integer)
(B) size_t
(C) void*
(D) double

13. Which arithmetic operation between two pointers is strictly ILLEGAL in C?

(A) Adding two pointers
(B) Subtracting two pointers
(C) Comparing pointers
(D) Incrementing pointer

14. Which heap allocation function allocates uninitialized contiguous memory bytes?

(A) malloc()
(B) calloc()
(C) realloc()
(D) free()

15. What does malloc() return if the operating system cannot satisfy the memory request?
- **(A)** NULL
- **(B)** 0xFFFFFFFF
- **(C)** Throws exception
- **(D)** -1

16. Which heap allocation function initializes all allocated bytes to zero (0)?
- **(A)** calloc()
- **(B)** malloc()
- **(C)** realloc()
- **(D)** valloc()

17. What are the two arguments required by calloc()?
- **(A)** (num_elements, size)
- **(B)** (total_bytes, 0)
- **(C)** (pointer, new_size)
- **(D)** (type, count)

18. Which standard library function resizes a previously allocated heap memory block?
- **(A)** realloc()
- **(B)** malloc()
- **(C)** calloc()
- **(D)** resize()

19. Which standard library function deallocates memory previously reserved on the heap?
- **(A)** free()
- **(B)** delete()
- **(C)** destroy()
- **(D)** release()

20. What header file must be included to use malloc(), calloc(), realloc(), and free()?
- **(A)** <stdlib.h>
- **(B)** <stdio.h>
- **(C)** <string.h>
- **(D)** <memory.h>

21. What is a pointer that continues to reference memory that has already been deallocated called?
- **(A)** Dangling pointer
- **(B)** Wild pointer
- **(C)** NULL pointer
- **(D)** Void pointer

22. What occurs when dynamically allocated heap memory is never freed before losing its pointer?
- **(A)** Memory leak
- **(B)** Stack overflow
- **(C)** Double free
- **(D)** Segmentation fault

23. What occurs when free() is executed twice on the exact same allocated heap pointer?
- **(A)** Double-free vulnerability
- **(B)** Memory leak
- **(C)** Silent success
- **(D)** Reallocation

24. What character terminates all standard character strings in C?
- **(A)** Null character ('\0')
- **(B)** Newline ('\n')
- **(C)** EOF (-1)
- **(D)** Period ('.')

25. What is the ASCII integer value of the null string terminator '\0'?
- **(A)** 0
- **(B)** 48
- **(C)** 32
- **(D)** -1

26. What header file must be included to access string manipulation functions like strlen, strcpy?
- **(A)** <string.h>
- **(B)** <strings.h>
- **(C)** <stdlib.h>
- **(D)** <stdio.h>

27. Which standard library function computes the length of a string excluding the null terminator?
- **(A)** strlen()
- **(B)** sizeof()
- **(C)** strsize()
- **(D)** strcount()

28. For char s[] = "Bio"; what are the values of strlen(s) and sizeof(s) respectively?
- **(A)** 3 and 4
- **(B)** 3 and 3
- **(C)** 4 and 4
- **(D)** 4 and 3

29. Which function copies a source string including null terminator into a destination buffer?

(A) strcpy()
(B) strncpy()
(C) memcpy()
(D) strcat()

30. Which safer string copy function specifies the maximum number of characters to copy?

(A) strncpy()
(B) strcpy()
(C) strdup()
(D) strcat()

31. Which function concatenates (appends) a source string onto the end of a destination string?

(A) strcat()
(B) strcpy()
(C) strncat()
(D) append()

32. Which function performs lexicographical comparison of two strings in C?

(A) strcmp()
(B) strcasecmp()
(C) strequal()
(D) strncmp()

33. What does strcmp(s1, s2) return when string s1 is identical to string s2?

(A) 0 (zero)
(B) 1
(C) -1
(D) Boolean true

34. What does strcmp(s1, s2) return when s1 is lexicographically smaller than s2?

(A) Negative integer (< 0)
(B) 0
(C) Positive integer (> 0)
(D) Null

35. Which function searches for the first occurrence of a character in a string?

(A) strchr()
(B) strrchr()
(C) strstr()
(D) strpbrk()

36. Which function searches for the last occurrence of a character in a string?

(A) strrchr()
(B) strchr()
(C) strstr()
(D) strtok()

37. Which function searches for the first occurrence of a substring needle within haystack?

(A) strstr()
(B) strchr()
(C) strtok()
(D) strspn()

38. What does strstr() return if the target substring is not found?

(A) NULL
(B) -1
(C) 0
(D) Empty string

39. Which function sets all bytes in a specified memory block to a given value?

(A) memset()
(B) memcpy()
(C) memmove()
(D) bzero()

40. Which memory copy function safely handles overlapping source and destination memory regions?

(A) memmove()
(B) memcpy()
(C) strcpy()
(D) memcmp()

41. Where are string literals like char *p = "DNA"; typically stored in memory?

(A) Read-only data segment
(B) Call stack
(C) Heap segment
(D) BSS segment

42. What happens if a program attempts to modify a string literal: char *p = "DNA"; p[0] = 'R';?

(A) Bus error / Seg fault
(B) Modifies character
(C) Syntax error
(D) Warning only

43. Where is the array stored in: char s[] = "DNA"; s[0] = 'R'; and is it mutable?
- **(A)** Call stack (mutable)
- **(B)** Read-only segment
- **(C)** Heap segment
- **(D)** BSS segment

44. What is the difference between const int *ptr and int * const ptr?
- **(A)** ptr to const vs const ptr
- **(B)** Both identical
- **(C)** int is constant
- **(D)** Pointer to pointer

45. In const int *ptr, can you modify the value pointed to (*ptr = 10)?
- **(A)** No (data is const)
- **(B)** Yes
- **(C)** Only if cast
- **(D)** Implementation defined

46. In int * const ptr = &x; can you reassign the pointer to another address (ptr = &y)?
- **(A)** No (pointer is const)
- **(B)** Yes
- **(C)** Only in C99
- **(D)** Only if x == y

47. What does const int * const ptr declare?
- **(A)** Immutable ptr to const data
- **(B)** Array of pointers
- **(C)** Void pointer
- **(D)** Dangling pointer

48. What is a pointer to a pointer in C?
- **(A)** Holds address of pointer
- **(B)** Array of structs
- **(C)** Void function
- **(D)** Memory block

49. What is the syntax to declare a double pointer to an integer?
- **(A)** int **pp;
- **(B)** int *pp*;
- **(C)** int &&pp;
- **(D)** int *(*pp);

50. How do you access the original integer value through double pointer int **pp?
- **(A)** **pp
- **(B)** *pp
- **(C)** &pp
- **(D)** *&pp

51. What does the declaration int *arr[10]; represent?
- **(A)** Array of 10 pointers to int
- **(B)** Pointer to array of 10 ints
- **(C)** Function pointer
- **(D)** 10-element int array

52. What does the declaration int (*ptr)[10]; represent?
- **(A)** Pointer to array of 10 ints
- **(B)** Array of 10 pointers
- **(C)** Pointer to pointer
- **(D)** Function returning array

53. What is the difference in pointer arithmetic between ptr[10] array and (*ptr)[10]?
- **(A)** ptr+1 skips 10 ints
- **(B)** ptr+1 skips 1 int
- **(C)** Identical offset
- **(D)** Syntax error

54. How is a dynamic 2D array allocated on the heap in C?
- **(A)** malloc row ptrs then cols
- **(B)** Single 1D malloc only
- **(C)** calloc with 2 dims
- **(D)** Impossible in C

55. In freeing a dynamically allocated 2D array (int **mat), what is the correct order?
- **(A)** Free rows then free mat
- **(B)** Free mat then free rows
- **(C)** Single free(mat)
- **(D)** Free in any order

56. Why is free(mat) followed by free(mat[i]) a severe bug?
- **(A)** Use-after-free corruption
- **(B)** Memory leak
- **(C)** Syntax error
- **(D)** Double free

57. What does realloc(ptr, 0) do in standard ANSI C?

(A) Equivalent to free(ptr)
(B) Allocates 1 byte
(C) Returns NULL leak
(D) Throws error

58. What does realloc(NULL, size) do?

(A) Equivalent to malloc(size)
(B) Throws error
(C) Returns NULL
(D) Frees memory

59. What bug occurs if you write: ptr = realloc(ptr, new_size); without a temporary pointer?

(A) Memory leak if realloc fails
(B) Double free
(C) Buffer overflow
(D) Syntax error

60. Why does ptr = realloc(ptr, new_size) leak memory when realloc fails and returns NULL?

(A) Original address is lost
(B) OS frees memory
(C) Pointer is corrupted
(D) Heap corrupted

61. How should realloc return value be handled defensively?

(A) Assign to temp, check NULL
(B) Assert not NULL
(C) Cast to void*
(D) Wrap in try-catch

62. Which function breaks a string into a sequence of tokens separated by delimiter characters?

(A) strtok()
(B) strstr()
(C) strchr()
(D) strsep()

63. How does strtok() mark the boundary of each extracted token in the source string?

(A) Overwrites delim with '\0'
(B) Allocates new string
(C) Returns index offset
(D) Pushes to stack

64. What argument must be passed as the first parameter to strtok() in subsequent tokenization calls?

(A) NULL
(B) Same string pointer
(C) Delimiter string
(D) EOF

65. Why is strtok() non-reentrant and dangerous in multi-threaded programming?

(A) Uses internal static state
(B) Allocates heap memory
(C) Modifies read-only memory
(D) Too slow

66. What POSIX function is the thread-safe, reentrant equivalent of strtok()?

(A) strtok_r()
(B) strtok_s()
(C) strsep()
(D) str_split()

67. In bioinformatics, which restriction enzyme recognizes the palindromic hexamer 5'-GAATTC-3'?

(A) EcoRI
(B) BamHI
(C) HindIII
(D) TaqI

68. Which restriction enzyme recognizes the hexamer 5'-GGATCC-3'?

(A) BamHI
(B) EcoRI
(C) HindIII
(D) NotI

69. Which restriction enzyme recognizes 5'-AAGCTT-3'?

(A) HindIII
(B) EcoRI
(C) BamHI
(D) AluI

70. How can strstr() be used to scan DNA for restriction enzyme cleavage sites?

(A) Returns pointer to match
(B) Counts total matches
(C) Splits sequence
(D) Replaces with gaps

71. What does strncpy(dest, src, n) NOT guarantee if length of src >= n?

(A) Null termination '\0'
(B) Copying all chars
(C) Buffer overflow check
(D) Fast execution

72. How do you guarantee null-termination when using strncpy(dest, src, sizeof(dest))?

(A) dest[sizeof(dest)-1]='\0'
(B) Use strcat after
(C) Call memset first
(D) Cast to char*

73. What is the result of applying sizeof to a pointer allocated via malloc(100 * sizeof(int))?

(A) sizeof(int*) [4 or 8]
(B) 400 bytes
(C) 100
(D) 0

74. Why cannot the C runtime determine the allocated size of a heap pointer via sizeof?

(A) sizeof is compile-time
(B) Pointer size is 0
(C) Heap is untracked
(D) Compiler bug

75. What standard function duplicates a string by allocating heap memory with malloc?

(A) strdup() (POSIX / C23)
(B) strcpy()
(C) strcopy()
(D) strclone()

76. What is the difference between memcpy() and memmove() when source and destination overlap?

(A) memmove safely buffers
(B) memcpy is safe
(C) memmove is slower O(N^2)
(D) Both identical

77. In manual implementation of strlen(const char *s), what loop condition detects string end?

(A) *s != '\0'
(B) *s > 0
(C) s != NULL
(D) sizeof(*s) != 0

78. In manual implementation of strcpy(char *d, const char *s), what is the canonical while idiom?

(A) while((*d++ = *s++));
(B) while(*s) *d=*s;
(C) while(*d == *s);
(D) for(;;*d++=*s++);

79. Why does while((*d++ = *s++)); successfully terminate after copying the null character?

(A) Assigns '\0' (0=false)
(B) Throws EOF signal
(C) Checks pointer bounds
(D) Post-increment stops

80. In manual strcmp(const char *s1, const char *s2), why must characters be cast to unsigned char?

(A) Prevents negative promotion
(B) ASCII is 7-bit only
(C) Char is 16-bit
(D) Required by GCC

81. What security vulnerability is known as 'Use-After-Free' (UAF)?

(A) Accessing memory post free()
(B) Calling free() twice
(C) Buffer overflow in heap
(D) Memory leak in loop

82. How do professional C programmers neutralize dangling pointers immediately after calling free(p)?

(A) p = NULL;
(B) p = 0xFFFFFFFF;
(C) memset(p, 0, 1);
(D) free(&p);

83. Why is free(NULL) completely safe and guaranteed to perform no operation in ANSI C?

(A) Guaranteed by ISO C spec
(B) Compiler optimization
(C) Null pointer exception
(D) Warning emitted

84. What is Heap Fragmentation in long-running C bioinformatics server applications?

(A) Scattered unusable holes
(B) Memory running out
(C) Stack corruption
(D) Cache invalidation

85. How does a Custom Memory Arena (Memory Pool) prevent heap fragmentation?

(A) Bulk allocation, bump ptr
(B) Calls malloc per byte
(C) Uses static arrays only
(D) Compresses RAM

86. In an arena bump allocator, how is deallocation performed?

(A) Reset offset to zero
(B) Call free per chunk
(C) Iterate linked list
(D) Invoke OS garbage coll

87. What dynamic programming table dimension is required to tokenize a FASTA genome of 10^7 bp?

(A) Requires block streaming
(B) 10^7 x 10^7 matrix
(C) Stack VLA allocation
(D) Static 2D array

88. In parsing PDB (Protein Data Bank) ATOM records, which function extracts formatted coordinates?

(A) sscanf()
(B) strtok()
(C) atoi()
(D) strcat()

89. What does sscanf(line, "ATOM %d %s %s %d %lf %lf %lf", ...) accomplish?

(A) Parses formatted string
(B) Prints to console
(C) Allocates heap struct
(D) Encrypts record

90. What is the return value of sscanf() upon complete matching of all 7 fields?

(A) 7
(B) 0
(C) 1
(D) -1

91. What is the difference between shallow copy and deep copy when copying structures with pointers?

(A) Deep allocates fresh memory
(B) Shallow copies memory
(C) Both duplicate buffer
(D) Deep is faster

92. Why does shallow copying a structure containing a heap pointer cause a double-free bug?

(A) Both structs share same ptr
(B) Memory is uninitialized
(C) Pointer is NULL
(D) Structure padding

93. What standard C library tool or flag detects memory leaks and heap buffer overflows during testing?

(A) AddressSanitizer (-fsanitize=address)
(B) GDB only
(C) Valgrind only
(D) Both A and C

94. What GCC compiler flag enables AddressSanitizer (ASan)?

(A) -fsanitize=address
(B) -fcheck-memory
(C) -Wall-memory
(D) -O3-sanitize

95. What is alignment padding in dynamic memory allocations?

(A) Rounding size to word boundary
(B) Adding zeroes to strings
(C) Stack realignment
(D) Bit field shifting

96. In standard 64-bit systems, malloc() returns memory aligned to at least how many bytes?

(A) 16 bytes (or 8 bytes)
(B) 1 byte
(C) 4 bytes
(D) 64 bytes

97. What function introduced in C11 allocates dynamically aligned memory to a specified boundary?

(A) aligned_alloc()
(B) posix_memalign()
(C) malloc_aligned()
(D) calloc_align()

98. What is the consequence of casting an unaligned char pointer to int* and dereferencing on ARM architectures?

(A) Hardware alignment fault
(B) Silent success always
(C) Truncation of bytes
(D) Automatic correction

99. What standard string function computes the length of the initial substring consisting only of accepted characters?

(A) strspn()
(B) strcspn()
(C) strpbrk()
(D) strstr()

100. What function computes the length of the initial substring consisting entirely of characters NOT in reject set?

(A) strcspn()
(B) strspn()
(C) strchr()
(D) strtok()

SECTION B: DESCRIPTIVE & SUBJECTIVE QUESTIONS

Q1. Explain Pointer Fundamentals in C, detailing address-of (&) and indirection (*) operators, pointer sizing, and physical memory addressing.

Q2. Detail Pointer Arithmetic rules in C, explaining scaling by sizeof(*ptr), pointer difference (ptrdiff_t), and illegal operations.

Q3. Explain the equivalence between Arrays and Pointers in C, detailing array decay, ptr[i] vs *(ptr + i), and the 2[arr] curiosity.

Q4. Explain Pointer Classification: Wild Pointers, NULL Pointers, Dangling Pointers, and Void Pointers with prevention strategies.

Q5. Contrast Pointers to Constants vs Constant Pointers vs Constant Pointers to Constants with syntax and code examples.

Q6. Detail Dynamic Memory Allocation functions: malloc(), calloc(), realloc(), and free(), explaining heap internals and failure handling.

Q7. Write an ANSI C program to allocate, populate, and deallocate a dynamic 2D array (matrix) using pointers to pointers (int **).

Q8. Explain Strings in C, contrasting string literals vs character arrays, null termination ('\0'), and string literal immutability.

Q9. Implement custom versions of core string library functions without <string.h>: my_strlen, my_strcpy, my_strcmp, and my_strcat.

Q10. Write an ANSI C program to search a DNA genomic string for Restriction Endonuclease recognition motifs using strstr().

Q11. Explain string tokenization using strtok(), detailing how delimiter replacement works, reentrancy limitations, and strtok_r().

Q12. Detail common Dynamic Memory Allocation bugs in C: Memory Leaks, Double Free, Use-After-Free, and Heap Buffer Overflows.

Q13. Write an ANSI C program to dynamically allocate a variable-length DNA sequence buffer, dynamically resizing it using realloc().

Q14. Write an ANSI C program to parse Protein Data Bank (PDB) ATOM record coordinates using sscanf() and pointer arithmetic.

Q15. Explain the difference between memcpy() and memmove(), detailing memory overlap handling and buffer safety.

Q16. Write an ANSI C program to reverse words in a sentence string in-place using two-pointer string reversal.

Q17. Explain memory layout of a compiled C program: Text, Data, BSS, Heap, and Stack segments with virtual address directionality.

Q18. Write an ANSI C program to implement a Custom Fixed-Size Memory Arena (Bump Allocator) for ultra-fast allocations without fragmentation.

Q19. Write an ANSI C program to check whether a given string is a Palindrome ignoring case and whitespace.

Q20. Explain deep copy vs shallow copy in C structures containing dynamic pointers, detailing memory aliasing and corruption traps.

Q21. Write an ANSI C program to count occurrences of a substring within a DNA sequence using pointer arithmetic and strstr().

Q22. Explain the restrict type qualifier in C99, detailing compiler pointer aliasing assumptions and optimization opportunities.

Q23. Write an ANSI C program to strip leading and trailing whitespace from a string in-place using pointer manipulation.

Q24. Explain pointer-to-pointer indirection mechanics, demonstrating how a function can modify the caller's pointer address.

Q25. Write an ANSI C program to sort an array of strings alphabetically using qsort() and pointer comparison callbacks.

Q26. Explain AddressSanitizer (ASan) and Valgrind tools for detecting C pointer corruption, memory leaks, and out-of-bounds errors.

Q27. Write an ANSI C program to concatenate two strings dynamically without using strcat() or standard library functions.

Q28. Explain pointer type casting and aliasing rules, detailing strict aliasing violations and the char* exception in ANSI C.

Q29. Write an ANSI C program to implement a generic memory swap function using void* and memcpy().

Q30. Write an ANSI C program to tokenize and extract sequence metadata from a Multi-FASTA file header string.

SECTION C: ANSWER KEY - MULTIPLE CHOICE QUESTIONS

Q1:	[A]	**Q21:**	[A]	**Q41:**	[A]	**Q61:**	[A]	**Q81:**	[A]
Q2:	[A]	**Q22:**	[A]	**Q42:**	[A]	**Q62:**	[A]	**Q82:**	[A]
Q3:	[A]	**Q23:**	[A]	**Q43:**	[A]	**Q63:**	[A]	**Q83:**	[A]
Q4:	[A]	**Q24:**	[A]	**Q44:**	[A]	**Q64:**	[A]	**Q84:**	[A]
Q5:	[A]	**Q25:**	[A]	**Q45:**	[A]	**Q65:**	[A]	**Q85:**	[A]
Q6:	[A]	**Q26:**	[A]	**Q46:**	[A]	**Q66:**	[A]	**Q86:**	[A]
Q7:	[A]	**Q27:**	[A]	**Q47:**	[A]	**Q67:**	[A]	**Q87:**	[A]
Q8:	[A]	**Q28:**	[A]	**Q48:**	[A]	**Q68:**	[A]	**Q88:**	[A]
Q9:	[A]	**Q29:**	[A]	**Q49:**	[A]	**Q69:**	[A]	**Q89:**	[A]
Q10:	[A]	**Q30:**	[A]	**Q50:**	[A]	**Q70:**	[A]	**Q90:**	[A]
Q11:	[A]	**Q31:**	[A]	**Q51:**	[A]	**Q71:**	[A]	**Q91:**	[A]
Q12:	[A]	**Q32:**	[A]	**Q52:**	[A]	**Q72:**	[A]	**Q92:**	[A]
Q13:	[A]	**Q33:**	[A]	**Q53:**	[A]	**Q73:**	[A]	**Q93:**	[D]
Q14:	[A]	**Q34:**	[A]	**Q54:**	[A]	**Q74:**	[A]	**Q94:**	[A]
Q15:	[A]	**Q35:**	[A]	**Q55:**	[A]	**Q75:**	[A]	**Q95:**	[A]
Q16:	[A]	**Q36:**	[A]	**Q56:**	[A]	**Q76:**	[A]	**Q96:**	[A]
Q17:	[A]	**Q37:**	[A]	**Q57:**	[A]	**Q77:**	[A]	**Q97:**	[A]
Q18:	[A]	**Q38:**	[A]	**Q58:**	[A]	**Q78:**	[A]	**Q98:**	[A]
Q19:	[A]	**Q39:**	[A]	**Q59:**	[A]	**Q79:**	[A]	**Q99:**	[A]
Q20:	[A]	**Q40:**	[A]	**Q60:**	[A]	**Q80:**	[A]	**Q100:**	[A]

MCQ Scoring Guide & Difficulty Distribution:

Questions 1 to 40: Foundational & Conceptual [Easy] | Questions 41 to 75: Analytical & Applied [Moderate]

Questions 76 to 100: Complex Numerical & Advanced Problem Solving [Difficult]

SECTION B: DETAILED SOLUTIONS FOR SUBJECTIVE QUESTIONS

1. **Explain the concept of pointers and their relationship with memory addresses.**
   A pointer is a variable that stores the memory address of another variable. Every variable in a program is stored at a unique location in memory, and that location has a numerical address. A pointer "points to" that address, allowing indirect access to the value stored there. The relationship is established using the address-of operator (`&`) and the indirection operator (`*`).

>>**Solution to Question 2:** Detail Pointer Arithmetic rules in C, explaining scaling by sizeof(*ptr), pointer difference (ptrdiff_t), and illegal operations.

>>Solution to Question 5: Main Pointer Classifications: Wild Pointers, Null Pointers, Dangling Pointers, and Void Pointers with brief explanations.

2. NULL Pointer:
- Definition: A pointer explicitly assigned the constant value NULL ((void*)0).
- Guarantee: Standard guarantees NULL points to an invalid address that cannot be dereferenced by any user program.
- Purpose: Serves as a defensive sentinel indicating 'points to nothing' or 'allocation failed'.

3. Dangling Pointer:
- Definition: A pointer that continues referencing a memory location after the storage has been released or destroyed.
Occurs when:
a. Calling free(ptr) without resetting ptr to NULL.
b. Returning the address of a local stack variable from a function.
- Hazard: 'Use-After-Free' security exploit. Memory may be reallocated to another process component; reading/writing it causes severe data corruption.
- Prevention: Immediately set pointer to NULL after freeing: free(p); p = NULL;

4. Void Pointer (Generic Pointer void*):
- Definition: A pointer that represents a raw memory address without any associated data type.
- Properties: Can hold address of any type (int*, double*, struct*).
- Dereference Restriction: CANNOT be dereferenced directly without explicit casting: *(int*)vp.
- Arithmetic Restriction: Standard C forbids pointer arithmetic on void* (since sizeof(void) is incomplete). Used extensively in malloc, qsort, and memcpy.

>> **Solution to Question 5:** Contrast Pointers to Constants vs Constant Pointers vs Constant Pointers to Constants with syntax and code examples.

```
int main(void) {
int a = 10, b = 20;

/* 1. Pointer to constant data */
const int *p1 = &a;
p1 = &b; /* LEGAL: Pointer can re-bind */
/* *p1 = 99; */ /* ILLEGAL: Data is read-only */

/* 2. Constant pointer */
int * const p2 = &a;
*p2 = 99; /* LEGAL: Data is mutable */
/* p2 = &b; */ /* ILLEGAL: Pointer cannot re-bind */

/* 3. Constant pointer to constant data */
const int * const p3 = &a;
/* *p3 = 50; */ /* ILLEGAL */
/* p3 = &b; */ /* ILLEGAL */

printf("Const pointer demonstrations verified.\n");
return 0;
}
```

>>Solution to Question 6: Detail Dynamic Memory Allocation functions: malloc(), calloc(), realloc(), and free(), explaining heap internals and failure handling.

```
int main(void) {
int initial_cap = 5;
int expanded_cap = 10;

/* 1. Allocate initial zeroed array */
int *gene_ids = (int *)calloc(initial_cap, sizeof(int));
if (gene_ids == NULL) {
fprintf(stderr, "Error: Heap allocation failed!\n");
return EXIT_FAILURE;
}

for (int i = 0; i < initial_cap; i++) gene_ids[i] = (i + 1) * 100;

/* 2. Defensive reallocation using temporary pointer */
int *temp = (int *)realloc(gene_ids, expanded_cap * sizeof(int));
if (temp == NULL) {
fprintf(stderr, "Error: Reallocation failed, preserving original block.\n");
free(gene_ids);
return EXIT_FAILURE;
}
gene_ids = temp; /* Safe update */

for (int i = initial_cap; i < expanded_cap; i++) gene_ids[i] = (i + 1) * 100;

printf("Dynamically expanded array (%d elements): ", expanded_cap);
for (int i = 0; i < expanded_cap; i++) printf("%d ", gene_ids[i]);
printf("\n");

/* 3. Deallocate and nullify dangling pointer */
free(gene_ids);
gene_ids = NULL;

return EXIT_SUCCESS;
}
```

>>Solution to Question 7: Write an ANSI C program to allocate, populate, and deallocate a dynamic 2D array (matrix) using pointers to pointers (int **).

3. Proper Deallocation Sequence:
Crucial Rule: MUST FREE IN STRICT REVERSE ORDER OF ALLOCATION!
Step 1: Loop through each row and free row buffer: free(mat[i]);
Step 2: Free the top-level pointer array: free(mat);
If free(mat) is executed first, pointers to row buffers are lost, permanently leaking heap memory!

4. Complete ANSI C Source Code:

```
#include <stdio.h>
#include <stdlib.h>

int **create_matrix(int rows, int cols) {
int **mat = (int **)malloc(rows * sizeof(int *));
if (!mat) return NULL;

for (int i = 0; i < rows; i++) {
mat[i] = (int *)malloc(cols * sizeof(int));
if (!mat[i]) {
/* Rollback previously allocated rows on failure */
for (int k = 0; k < i; k++) free(mat[k]);
free(mat);
return NULL;
}
}
return mat;
}

void destroy_matrix(int **mat, int rows) {
if (!mat) return;
for (int i = 0; i < rows; i++) {
free(mat[i]); /* Free individual column arrays */
}
free(mat); /* Free row pointer array */
}

int main(void) {
int r = 3, c = 4;
int **grid = create_matrix(r, c);
if (!grid) return 1;

/* Populate matrix */
for (int i = 0; i < r; i++) {
for (int j = 0; j < c; j++) grid[i][j] = (i + 1) * 10 + j;
}

printf("Dynamic 2D Matrix Content:\n");
for (int i = 0; i < r; i++) {
for (int j = 0; j < c; j++) printf("%4d ", grid[i][j]);
printf("\n");
}

destroy_matrix(grid, r);
grid = NULL;
```

```
  printf("Matrix successfully freed without memory leaks.\n");

  return 0;
  }
```

>>Solution to Question 8: Implement a custom version of `strcat` called `my_strcat`. This function should append the source string to the destination string without using any standard library functions. Ensure that the destination string has enough memory allocated to hold the concatenated result.

[Detailed Step-by-Step Answer]:
Custom Implementations of Core C String Functions:
Writing standard string functions from scratch demonstrates fundamental pointer manipulation and null-termination mechanics:

1. Custom String Length (my_strlen):

```
size_t my_strlen(const char *s) {
const char *p = s;
while (*p != '\0') p++;
return (size_t)(p - s); /* Pointer subtraction */
}
```

2. Custom String Copy (my_strcpy):

```
char *my_strcpy(char *dest, const char *src) {
char *orig_dest = dest;
/* Idiomatic C: copies char, evaluates value, terminates when '\0' is copied */
while ((*dest++ = *src++) != '\0');
return orig_dest;
}
```

3. Custom String Comparison (my_strcmp):

```
int my_strcmp(const char *s1, const char *s2) {
while (*s1 && (*s1 == *s2)) {
s1++;
s2++;
}
/* Cast to unsigned char to prevent negative char sign-extension bugs */
return (int)((unsigned char)*s1 - (unsigned char)*s2);
}
```

4. Custom String Concatenation (my_strcat):

```
char *my_strcat(char *dest, const char *src) {
char *p = dest;
while (*p != '\0') p++; /* Advance to null terminator of dest */
while ((*p++ = *src++) != '\0'); /* Append source */
return dest;
}
```

5. Complete Test Driver:

```
#include <stdio.h>

int main(void) {
char buf[64];
my_strcpy(buf, "Polymerase");
printf("my_strcpy: %s (Length: %zu)\n", buf, my_strlen(buf));

my_strcat(buf, "_Chain_Reaction");
printf("my_strcat: %s\n", buf);

int cmp = my_strcmp("DNA", "RNA");
printf("my_strcmp(\"DNA\", \"RNA\"): %d (Negative denotes DNA < RNA)\n", cmp);

return 0;
}
```

>>Solution to Question 10: an ANSI C program to search a DNA genomic string for Restriction Endonuclease recognition motifs using strstr(),

---

>>Solution to Question 14 Dynamic Memory Allocation in C: Memory Leaks, Double Free, Use After Free, and Heap Buffer Overflows.

The Four Critical Heap Memory Vulnerabilities in C:
1. Memory Leak:
- Cause: Allocating heap memory via malloc/calloc and allowing all pointers referencing that block to be overwritten or fall out of scope without calling free().
- Consequence: Heap memory grows monotonically, exhausting physical RAM and swap space, resulting in Out-Of-Memory (OOM) kernel panics.

2. Double-Free Vulnerability:
- Cause: Invoking free(ptr) twice on the exact same heap memory pointer without an intervening allocation.
- Consequence: Corrupts the heap allocator's internal free-list bin metadata (fastbins / tcache), enabling arbitrary code execution exploits.

3. Use-After-Free (UAF):
- Cause: Accessing memory through a pointer after calling free(ptr).
- Consequence: The deallocated memory may be reallocated to another object. Modifying the old pointer corrupts the new object's data or vtable pointers.

4. Heap Buffer Overflow:
- Cause: Writing more bytes into a dynamically allocated buffer than the requested allocation size.
- Consequence: Overwrites heap metadata chunks and adjacent allocated objects, destroying heap consistency.

5. Prevention Best-Practice Code:

```
#include <stdio.h>
#include <stdlib.h>

void safe_deallocate(void **ptr_ref) {
if (ptr_ref != NULL && *ptr_ref != NULL) {
free(*ptr_ref);
*ptr_ref = NULL; /* Nullifies dangling pointer */
}
}

int main(void) {
int *buffer = (int *)malloc(10 * sizeof(int));
/* Safe deallocation macro / wrapper */
safe_deallocate((void **)&buffer);

/* Second free is now completely safe (free(NULL) is no-op) */
safe_deallocate((void **)&buffer);
printf("Safely avoided double-free and dangling pointer traps.\n");
return 0;
}
```

>>Solution to Question 12: Write an ANSI C program to dynamically allocate a variable-length DNA sequence buffer, dynamically resizing it using realloc(),

unknown upfront.
Start with a modest buffer capacity. When capacity is reached, double the buffer size using realloc() (amortized O(1) growth strategy).

2. Complete ANSI C Source Code:

```
#include <stdio.h>
#include <stdlib.h>

#define INITIAL_CAPACITY 8

typedef struct {
char *data;
size_t length;
size_t capacity;
} DynamicDnaBuffer;

DynamicDnaBuffer *create_dna_buffer(void) {
DynamicDnaBuffer *buf = (DynamicDnaBuffer *)malloc(sizeof(DynamicDnaBuffer));
if (!buf) return NULL;

buf->data = (char *)malloc(INITIAL_CAPACITY * sizeof(char));
if (!buf->data) { free(buf); return NULL; }

buf->length = 0;
buf->capacity = INITIAL_CAPACITY;
buf->data[0] = '\0';
return buf;
}

int append_base(DynamicDnaBuffer *buf, char base) {
/* Check if capacity is reached (reserve 1 byte for null terminator) */
if (buf->length + 1 >= buf->capacity) {
size_t new_cap = buf->capacity * 2;
char *temp = (char *)realloc(buf->data, new_cap * sizeof(char));
if (!temp) return -1; /* Allocation failed */

buf->data = temp;
buf->capacity = new_cap;
printf("[Buffer Growth]: Resized capacity to %zu bytes\n", new_cap);
}

buf->data[buf->length++] = base;
buf->data[buf->length] = '\0';
return 0;
}

void free_dna_buffer(DynamicDnaBuffer *buf) {
if (buf) {
free(buf->data);
free(buf);
}
}

int main(void) {
```

```
DynamicDnaBuffer *stream = create_dna_buffer();
const char *simulated_stream = "ATGCGATCGATCGATCGCGCATATCGATCAGCTATCGAT";

for (int i = 0; simulated_stream[i] != '\0'; i++) {
append_base(stream, simulated_stream[i]);
}

printf("Final Sequence: %s\n", stream->data);
printf("Total Bases Captured: %zu | Final Buffer Capacity: %zu bytes\n",
stream->length, stream->capacity);

free_dna_buffer(stream);
return 0;
}
```

Solution to Question 14: an ANSI C program to parse Protein Data Bank (PDB) ATOM record coordinates using sscanf() and pointer arithmetic.

```
"ATOM 2 CA MET A 1 26.266 25.413 2.842",
"ATOM 3 C MET A 1 26.913 26.639 3.531",
"HETATM 1001 O HOH A 101 30.120 15.400 12.110"
};
int count = sizeof(pdb_lines) / sizeof(pdb_lines[0]);

printf("=== PDB Coordinate Parser ===\n");
for (int i = 0; i < count; i++) {
PdbAtom atom;
if (parse_pdb_atom_line(pdb_lines[i], &atom)) {
printf("Atom #%d [%-4s of %s %d-%c] -> X:%7.3f, Y:%7.3f, Z:%7.3f\n",
atom.atom_number, atom.atom_name, atom.res_name,
atom.res_seq, atom.chain, atom.x, atom.y, atom.z);
} else {
printf("Skipping Non-ATOM record: %-20.20s...\n", pdb_lines[i]);
}
}
return 0;
}
```

>> Solution to Question 15

Solution to Question 15

Explain the difference between memcpy() and memmove(), detailing memory overlap handling and buffer safety.

```
  char buf[16] = "ABCDEF";
  memmove(buf + 2, buf, 4); /* Shift right by 2 */
  printf("memmove shift right: %s\n", buf); /* Prints "ABABCD" */

  return 0;
  }
```

Solution to Question 16: Write an ANSI C program to reverse words in a sentence string in-place using two-pointer string reversal.

```
  printf("Original: %s\n", text);
  reverse_words(text);
  printf("Reversed: %s\n", text);
  return 0;
  }
```

Solution to Question 17: Explain memory layout of a compiled C program: Text, Data, BSS, Heap, and Stack segments with virtual address directionality.

>>Solution to Question 18: Write an ANSI C program to implement a Custom Fixed-Size Memory Arena (Bump Allocator) for ultra-fast allocations without fragment

```
  int *scores = (int *)arena_alloc(arena, 5 * sizeof(int));
  char *tag = (char *)arena_alloc(arena, 32);

  for (int i = 0; i < 5; i++) scores[i] = (i + 1) * 10;
  sprintf(tag, "Illumina_HiSeq_Read_01");

  printf("Arena Allocated Tag: %s\n", tag);
  printf("Arena Offset Used: %zu of %zu bytes\n", arena->offset, arena->capacity);

  arena_reset(arena);
  printf("Arena reset: offset reset to %zu bytes\n", arena->offset);

  destroy_arena(arena);
  return 0;
  }
```

Solution to Question 19: Write an ANSI C program to check whether a given string is a Palindrome ignoring case and whitespace.

```
return true;
}

int main(void) {
const char *phrases[] = {
"Racecar",
"A man, a plan, a canal, Panama!",
"Biotechnology",
"Was it a car or a cat I saw?"
};
int count = sizeof(phrases) / sizeof(phrases[0]);

for (int i = 0; i < count; i++) {
printf("\"%-35s\" -> %s\n", phrases[i],
is_palindrome_clean(phrases[i]) ? "PALINDROME" : "NOT Palindrome");
}
return 0;
}
```

>>Solution to Question 20 and 21: Convert sentences into a palindrome with DNA-themed twists, detailing memory allocation traps.

1. Complete ANSI C Source Code:

```
#include <stdio.h>
#include <string.h>

int count_substring_occurrences(const char *haystack, const char *needle) {
if (!haystack || !needle || *needle == '\0') return 0;

int count = 0;
int needle_len = strlen(needle);
const char *p = haystack;

while ((p = strstr(p, needle)) != NULL) {
count++;
p += needle_len; /* Advance past match (non-overlapping count) */
}
return count;
}

int main(void) {
const char *genome = "ATGAAATGCCCATGTTTATGCCATG";
const char *motif = "ATG";

int total = count_substring_occurrences(genome, motif);
printf("Genome: %s\n", genome);
printf("Total Start Codon (ATG) occurrences: %d\n", total);

return 0;
}
```

>> Solution to Question 22

>> Solution to Question 23 Explain the restrict type qualifier in C99, detailing compiler pointer aliasing assumptions and optimization opportunities.

>>Solution Question 28 ANS: To pointer to strict in the and is the instance from a function and is the pointer's memory address.

passes the POINTER BY VALUE (a local copy of the pointer address is created).
Updating 'p' modifies ONLY the local copy; the caller's original pointer remains unchanged (dangling/NULL)!

2. The Solution:
Pass the MEMORY ADDRESS of the pointer variable (a double pointer Type **):

```
void allocate_safe(int **pp) {
*pp = (int *)malloc(100 * sizeof(int));
}
```

Dereferencing *pp modifies the caller's actual pointer variable directly!

3. Demonstrative ANSI C Program:

```
#include <stdio.h>
#include <stdlib.h>

void allocate_buffer(char **buf_ptr, int size) {
*buf_ptr = (char *)malloc(size * sizeof(char));
}

int main(void) {
char *data = NULL;
allocate_buffer(&data, 64);

sprintf(data, "Dynamic Buffer Allocated via Double Pointer");
printf("Result: %s\n", data);

free(data);
return 0;
}
```

>>Solution to Question 25: Write an ANSI C program to sort an array of strings alphabetically using qsort() and pointer comparison callbacks.

```
int main(void) {
const char *enzymes[] = {
"TaqPolymerase",
"EcoRI",
"Cas9",
"BamHI",
"DNA_Ligase",
"HindIII"
};
int count = sizeof(enzymes) / sizeof(enzymes[0]);

qsort(enzymes, count, sizeof(const char *), compare_strings);

printf("=== Alphabetically Sorted Reagents ===\n");
for (int i = 0; i < count; i++) {
printf("%d. %s\n", i + 1, enzymes[i]);
}
return 0;
}
```

>>Solution to Question 28 Compile with -fsanitize=address and valgrind tools for dynamic analysis. Use static analysis tools like cppcheck or clang-tidy. Avoid unsafe functions like gets() or strcpy() without bounds checks.

```
while (s1[len1] != '\0') len1++;
while (s2[len2] != '\0') len2++;

char *result = (char *)malloc((len1 + len2 + 1) * sizeof(char));
if (!result) return NULL;

size_t i = 0;
for (size_t j = 0; j < len1; j++) result[i++] = s1[j];
for (size_t k = 0; k < len2; k++) result[i++] = s2[k];
result[i] = '\0';

return result;
}

int main(void) {
const char *genus = "Escherichia";
const char *species = "_coli_K12";

char *organism = dynamic_concat(genus, species);
printf("Concatenated Organism String: %s\n", organism);

free(organism);
return 0;
}
```

>>**Solution to Question 28:** An ANSI C program to implement rules to maintain valid function calls and handle exceptions in ANSI C.

By utilizing void* pointers and explicit element byte size, a single swap function can swap any arbitrary data type (int, double, structures):

2. Complete ANSI C Source Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

void generic_swap(void *a, void *b, size_t size) {
void *temp = malloc(size);
memcpy(temp, a, size);
memcpy(a, b, size);
memcpy(b, temp, size);
free(temp);
}

int main(void) {
double conc1 = 14.5, conc2 = 98.2;
printf("Before: conc1=%.1f, conc2=%.1f\n", conc1, conc2);
generic_swap(&conc1, &conc2, sizeof(double));
printf("After: conc1=%.1f, conc2=%.1f\n", conc1, conc2);
return 0;
}
```

>>Solution to Question 30: an ANSI C program to tokenize and extract sequence metadata from a Multi-FASTA file header string.

```
  printf("=== Extracted Genomic Metadata ===\n");
  printf("Accession ID: %s\n", accession);
  printf("Host Species: %s\n", organism);
  printf("Target Gene: %s\n", gene);

  return 0;
  }
```
